centro universitário FEI

CC1620 - Desenvolvimento Web
CCP120 - Programação Full Stack

Não esqueçam de marcar presença!

Mais sobre JavaScript



Algumas maneiras de se criar variáveis

Sem nenhuma palavra chave

- *variável global (não recomendado!)*

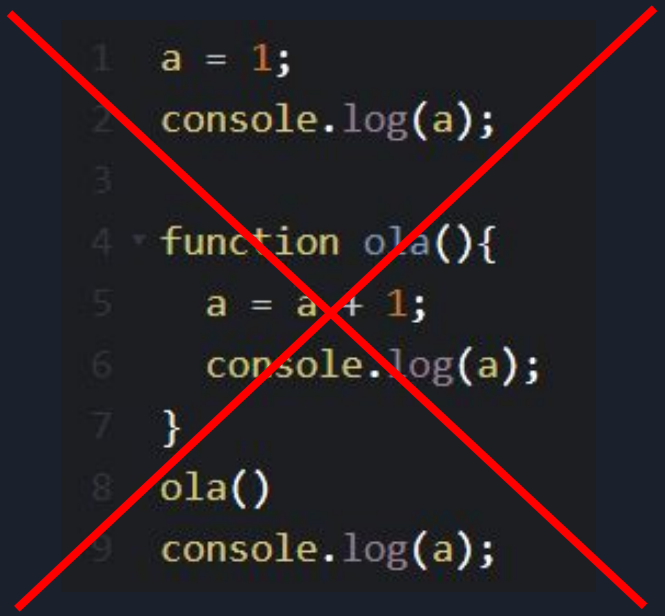
```
1  a = 1;
2  console.log(a);
3
4  function ola(){
5    a = a + 1;
6    console.log(a);
7  }
8  ola()
9  console.log(a);
```

Escopo da variável a

https://codepen.io/danilo-perico/pen/XWKWxEw?editors=0012

Sem nenhuma palavra chave

- *variável global (não recomendado!)*



```
a = 1;
console.log(a);

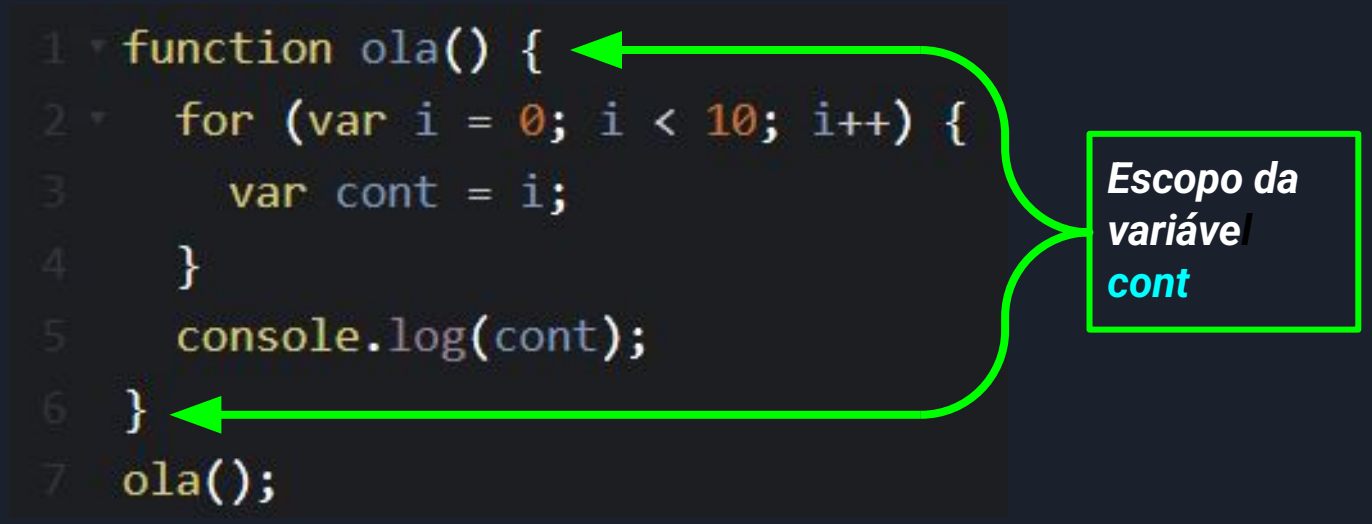
function ola(){
  a = a + 1;
  console.log(a);
}
ola()
console.log(a);
```

https://codepen.io/danilo-perico/pen/XWKWxEw?editors=0012

var

- *var - variável com escopo de função*

```
1 function ola() {
2   for (var i = 0; i < 10; i++) {
3     var cont = i;
4   }
5   console.log(cont);
6 }
7 ola();
```



https://codepen.io/danilo-perico/pen/gOMOBjx?editors=0012

let

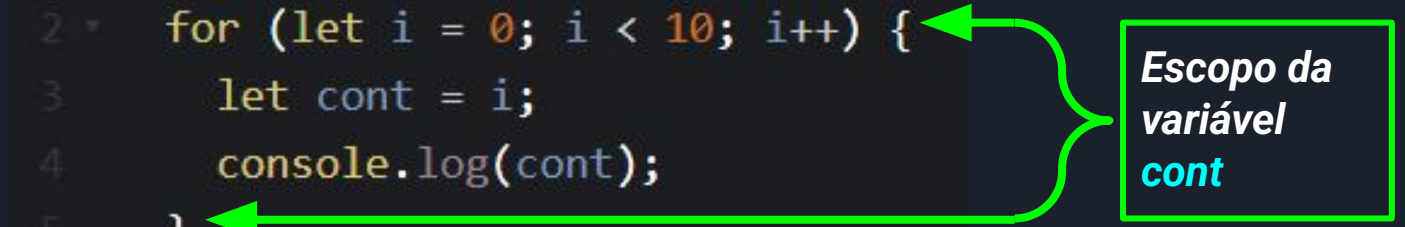
- ***let** - variável com escopo de bloco*

```
function ola() {
  for (let i = 0; i < 10; i++) {
    let cont = i;
  }
  console.log(cont);
Uncaught ReferenceError: cont is not defined
}
ola();
```

let

- *let - variável com escopo de bloco*

```
1  function ola() {
2    for (let i = 0; i < 10; i++) {
3      let cont = i;
4      console.log(cont);
5    }
6  }
7  ola();
```



https://codepen.io/danilo-perico/pen/dyXygEN?editors=0002

const

- *const - igual ao let, só que as variáveis são read-only*

```
1  const a = 1;
2  a = 2;
```

```
Uncaught TypeError: Assignment to constant variable.
```

JavaScript



- No JavaScript podemos utilizar também Vetores (Arrays) para armazenar múltiplos dados
- São equivalentes às listas em Python

Vetores (*Arrays*)

```
var idades = [ 18, 20]; // 18 , 20

idades.push(25); // 18 , 20, 25

idades.pop() // 18 , 20

delete idades[0] // 20
```

- Alguns métodos (funções) para *arrays* são:
  - *push* - adiciona um novo elemento no fim
  - *pop* - remove o último elemento
  - *delete* - remove elementos

https://www.w3schools.com/js/js_array_methods.asp

Objetos e Classes

Uma breve introdução a orientação a objeto.

Objetos

- Objetos representam itens do mundo real ou do domínio do sistema
- Logo, representam as *propriedades* e os *comportamentos* destes itens





Objetos

- Os comportamentos mapeados são chamados de **métodos**

Métodos são **funções** que estão dentro do objeto

- As propriedades são chamadas de **atributos**

Os atributos são as **variáveis** do objeto

Objetos

- Logo:
  - Os objetos são como variáveis que podem armazenar outras variáveis e/ou funções

Objetos

- Há diversas formas de se criar objetos no JavaScript!
- Uma delas e notação de objetos:

```
var meuObj = {
    idade: 5,
    altura: 1.75,
    imprimeIdade: function(){
        alert(this.idade);
    }
};
```

Exemplo 1 - Criando um Retângulo

- Quais são os possíveis atributos?

Exemplo 1 - Criando um Retângulo

- Quais são os possíveis atributos?
  - x
  - y
  - largura
  - altura
  - cor

Exemplo 1 - Criando um Retângulo

- Quais são os possíveis atributos?
  - x
  - y
  - largura
  - altura
  - cor

```
var retangulo = {
  x: 0,
  y: 0,
  largura: 30,
  altura: 20,
  cor: "red"
}
```

Exemplo 2 - Criando dois retângulos no mesmo programa

```
var retangulo1 = {
  x: 0,
  y: 0,
  largura: 30,
  altura: 20,
  cor: "red"
}
```

```
var retangulo2 = {
  x: 100,
  y: 100,
  largura: 30,
  altura: 20,
  cor: "blue"
}
```

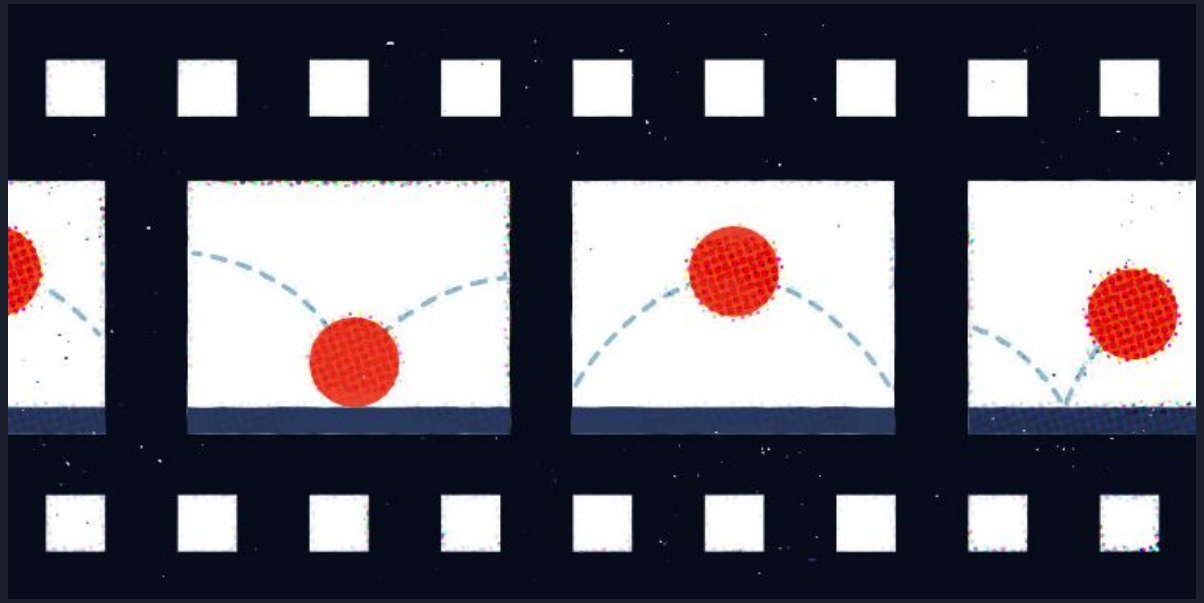


<canvas>

Animações

Canvas - animações

- Animações são baseadas em repetidas atualizações da posição de objetos na tela:



Canvas - animações

- Três funções podem ser utilizadas para isto:
  - *setInterval(funcao, tempo)*
    - Inicia repetidamente executando a função especificada a cada milésimo de segundo de atraso.
  - *setTimeout(funcao, tempo)*
    - Executa a função especificada com milissegundos de atraso.
  - *requestAnimationFrame(funcao)*
    - Informa ao navegador que você deseja executar uma animação e solicita que o navegador chame uma função específica para atualizar uma animação antes da próxima repintura.

Canvas - animações

- A melhor função para produzir uma animação:
  - *requestAnimationFrame(funcao)*
    - O método *requestAnimationFrame* fornece uma maneira mais suave e eficiente de animação, chamando o quadro de animação quando o sistema estiver pronto para pintar o quadro.
    - O número de retornos de chamada geralmente é de 60 vezes por segundo e pode ser reduzido a uma taxa mais baixa quando executado em segundo plano.

Exemplo

- Construir um elemento com as funções de desenho do canvas, e, então, fazer este elemento se movimentar para direita
- Utilize a ideia de objeto para construção do retângulo!

Exemplo de elemento retângulo

```
let retangulo = {
    x: 0,
    y: 0,
    altura: 50,
    largura: 50,
    cor: "red",
    desenha: function(){
        ctx.beginPath();
        ctx.fillStyle = this.cor;
        ctx.fillRect(this.x,this.y,this.largura,this.altura);
        ctx.closePath();
    }
}
```

Exemplo

```
function animacao(){
    ctx.clearRect(0,0,400,400)
    retangulo.x = retangulo.x + 1;
    retangulo.desenha();
    requestAnimationFrame(animacao)
}
animacao();
```

Canvas - animações

- O canvas permite também a interação do usuário, por meio de eventos
- Alguns eventos podem ser gerados pelo mouse, outros pelo teclado etc.
- Função: *document.addEventListener(“evento”, function)*

Canvas - animações

- Função *addEventListener(“evento”, function)*
- Exemplo - capturar quando uma tecla é pressionada:

```
document.addEventListener("keydown", function(evento) {
  tecla = evento.keyCode;
});
```

Exemplo

- Construir um elemento com as funções de desenho do canvas, e, então, fazer este elemento se movimentar seguindo o teclado
- Utilize a ideia de objeto para construção do retângulo!

Exemplo de elemento circulo

```
let elemento = {
    x: 0,
    y: 200,
    raio: 50,
    cor: "blue",
    desenha: function(){
        ctx.beginPath();
        ctx.fillStyle = this.cor;
        ctx.arc(this.x,this.y,this.raio,0,2*Math.PI);
        ctx.fill()
        ctx.closePath();
    }
}
```

Exemplo

```
function animacao(){
    ctx.clearRect(0,0,400,400)
    circulo.desenha();
    requestAnimationFrame(animacao)
}
animacao();
document.addEventListener('keydown',function(evento){
    tecla = evento.key;
    console.log(tecla);
    if(tecla == 'ArrowUp')    {circulo.y = circulo.y-1}
    if(tecla == 'ArrowDown')  {circulo.y = circulo.y+1}
    if(tecla == 'ArrowLeft')  {circulo.x = circulo.x-1}
    if(tecla == 'ArrowRight'){circulo.x = circulo.x+1}
})
```

Canvas - animações

- Função *addEventListener("evento", function)*
- Exemplo - capturar posição do mouse

```
addEventListener('mousemove', function(evento) {
  rect = canvas.getBoundingClientRect();
  x_mouse = evento.clientX - rect.left,
  y_mouse = evento.clientY - rect.top;
  console.log(x_mouse, y_mouse);
});
```

Exemplo de elemento imagem

```
let bola = {
    x: 0,
    y: 100,
    raio: 50,
    img: new Image(),
    desenha: function(){
        this.img.src = 'ball.jpg';
        ctx.beginPath();
        ctx.drawImage(this.img, this.x, this.y, 2*this.raio, 2*this.raio);
        ctx.closePath();
    }
}
```

Exemplo

```
function animacao(){
    ctx.clearRect(0,0,400,400)
    bola.desenha();
    requestAnimationFrame(animacao)
}
animacao();
document.addEventListener('mousemove',function(evento){
    let rect = canvas.getBoundingClientRect();
    let x_mouse = evento.clientX - rect.left;
    let y_mouse = evento.clientY - rect.top;
    console.log(x_mouse,y_mouse);
    bola.x = x_mouse;
    bola.y = y_mouse;
})
```

Exemplo de movimentação básica

https://replit.com/@daniloperico/canvasmovimentobasico

Vá além!

Leia mais sobre uso de Objetos em JavaScript

https://www.w3schools.com/js/js_object_definition.asp

Não esqueçam de marcar presença!

Material desenvolvido por:

- Danilo Périco
- Isaac de Jesus
- Fagner Pimentel

Fontes:

- https://www.w3schools.com/